AWS Cost Optimization

—> What is AWS Cost Optimization?

AWS Cost Optimization means **reducing unnecessary AWS spending** while still keeping:

- Good performance
- High availability
- Security

In simple words: **Use only what you need, pay less, and avoid waste.**

—> Why Cost Optimization is Important

- AWS is **pay-as-you-go** → wrong setup = high bill
- Helps companies **save money**
- Avoids **unused resources**
- Makes cloud usage **efficient**

—> Pillars of AWS Cost Optimization

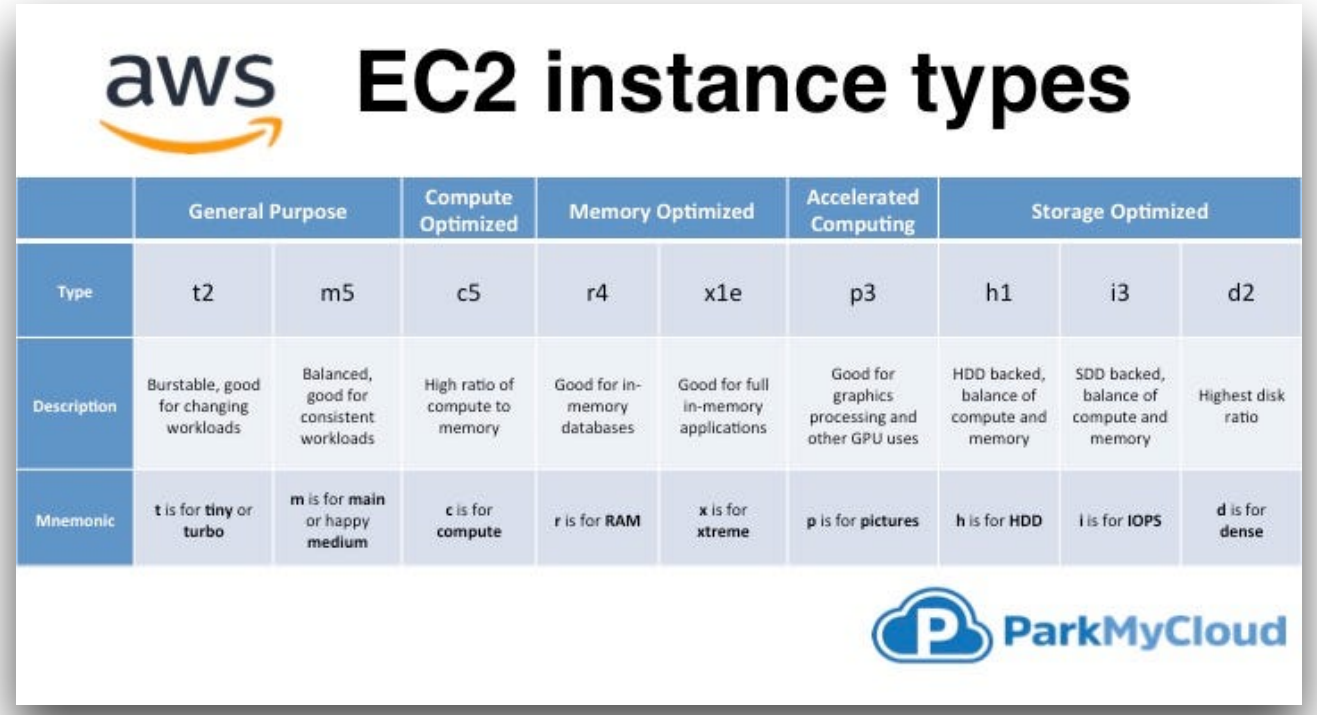
AWS follows **Cost Optimization pillar** from Well-Architected Framework.

Main goals:

- Right-size resources
- Use cheaper pricing models
- Remove unused services
- Monitor spending regularly

—> Common AWS Services That Increase Cost

- EC2 instances
- EBS volumes & snapshots
- Load Balancers
- RDS databases
- Data transfer
- CloudWatch logs



—> EC2 Cost Optimization

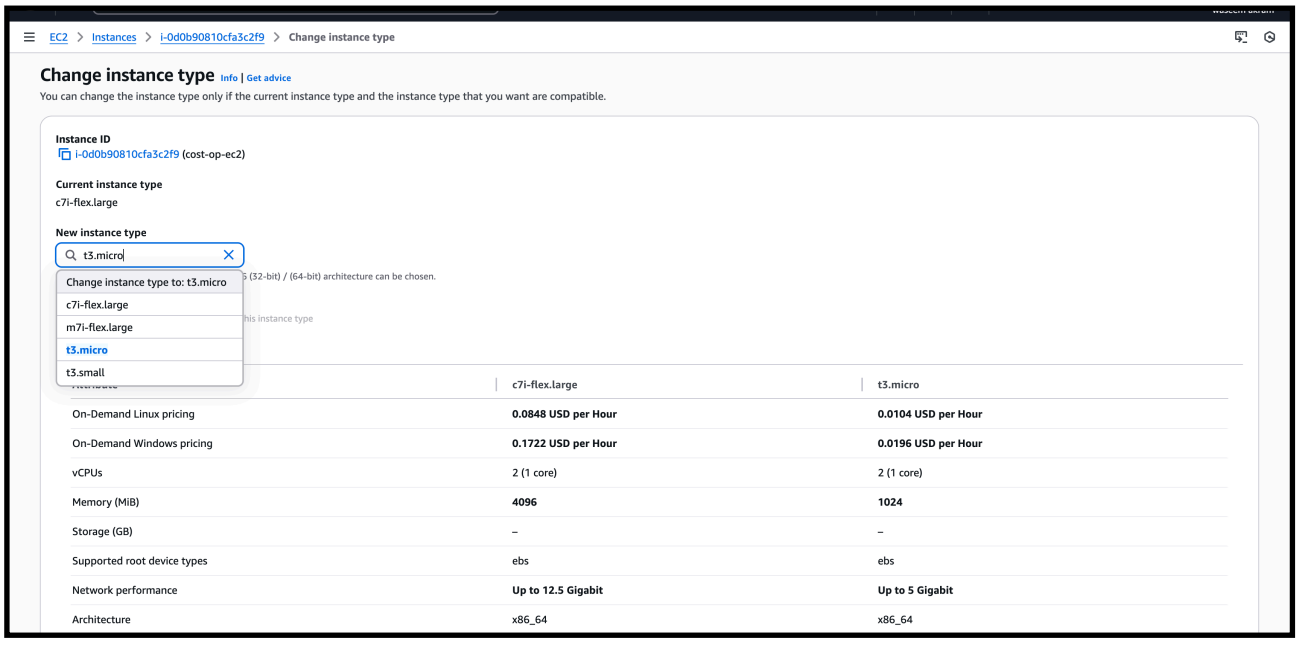
a) Right-Sizing EC2

- Do not use **large instance** if small is enough
- Example:
    - Instead of `t3.large` → use `t3.micro`

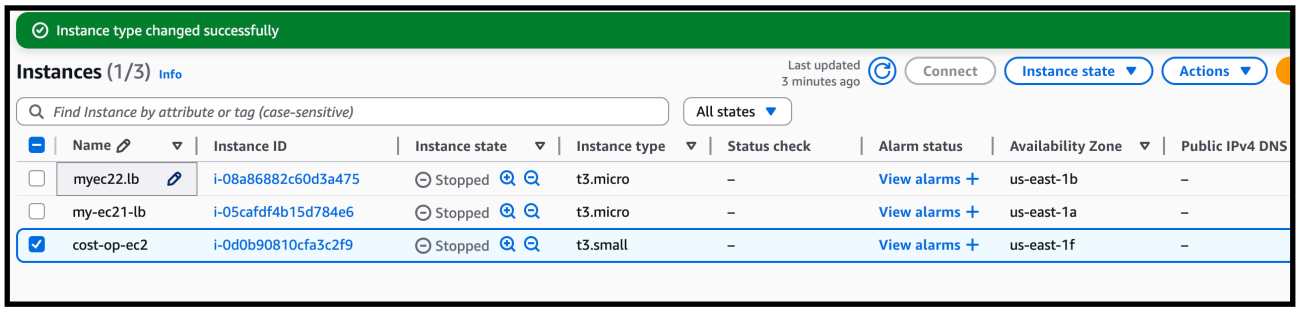
Right-sizing means **choosing the correct EC2 instance size** based on actual usage.

- Do not use a **large instance** if CPU and memory usage is low
- Check **CPU, memory, and network usage** before selecting instance size
- Oversized instances increase **monthly AWS bill** without benefit
- Smaller instances give **same performance** for low-traffic applications
- Best for:
    - Development servers
    - Testing environments
    - Small websites
- Always match instance size with **workload requirement**

Example : change the instance type of instance after stopping instance —>action —> change instance type



—> and save the changes this will optimise the cost of instance



- Result:
  - Same application works
  - **Less cost**
  - Better resource utilization

—> “Right-sizing EC2 means selecting the correct instance size so we don’t pay extra for unused CPU and memory.”

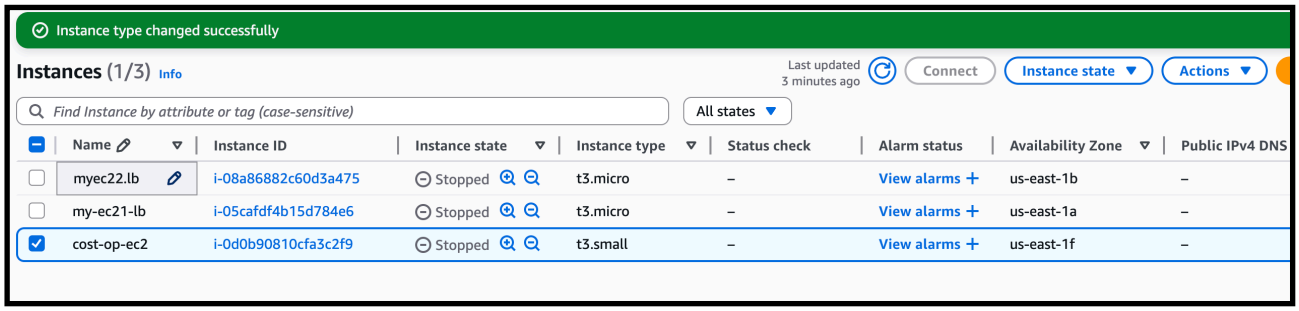
b) Stop Unused Instances

- Stop **dev/test** servers when not in use
- Stopped instance = **no compute cost**

—> Stopping unused EC2 instances helps **reduce unnecessary AWS cost**.

Points:

- Development and testing servers are **not required 24/7**
- Stop EC2 instances when:
  - Office hours are over
  - Work is completed
  - Application is not in use
- When an instance is **stopped**:
  - **No EC2 compute cost**
  - Only storage (EBS) cost is charged
- Very useful for:
  - Dev environments
  - Test environments
  - Temporary projects
- Restart instance anytime when needed
- Helps avoid **wasting money on idle servers**



Example:

- Dev EC2 runs from **10 AM – 6 PM**
- Stop instance after work hours
- Restart next day
- Result:
  - Same work
  - **Lower monthly AWS bill**

Best Practice:

- Create a habit of stopping instances daily
- Use automation to stop/start instances on schedule

—> “Stopping unused EC2 instances saves cost because AWS does not charge compute cost for stopped instances.”

c) Use Spot Instances

- Very cheap (up to 90% less)
- Best for:
  - Testing
  - Batch jobs
  - Non-critical workloads

—> EBS & Snapshot Cost Optimization

—>Delete **unused EBS volumes**

—>Remove **old snapshots**

—>Use **Lifecycle Manager** to auto-delete old snapshots

—>Choose correct volume type:

- gp3 instead of gp2 (cheaper)

a) Delete **unused EBS volumes**

Unused EBS volumes continue to **generate cost even if no EC2 instance is using them**.

When an EC2 instance is **terminated**, sometimes its EBS volume becomes **detached.**

Detached EBS volume means:

- No instance is using it
- Data is not required anymore

Even if the volume is unused, **AWS still charges for storage**

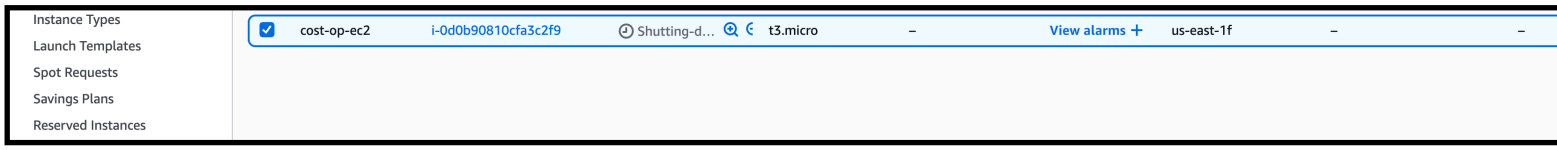
Such volumes should be **identified and deleted**

Deleting unused EBS volumes helps:

- Reduce storage cost
- Avoid unnecessary monthly billing

Example:

- EC2 instance is terminated
- Root or additional EBS volume becomes **detached**
- Volume is no longer required
- Delete the volume



- Result: "Deleting unused EBS volumes saves cost because AWS charges for storage even when the volume is not attached to any instance."
    - Storage removed
    - **EBS cost saved**

Best Practices:

- Regularly check for **available (detached) volumes**
- Delete volumes that are:
    - Not attached to any instance
    - Not needed for backup
- Keep only required volumes

B) Remove old snapshots

EBS snapshots are stored in S3 and **cost money as long as they exist**, even if the instance is deleted.

Points:

- Snapshots are usually taken for:
  - Backup
  - Recovery
- Over time, **old snapshots are no longer required**
- Even if:
  - EC2 instance is terminated
  - EBS volume is deleted
    —> Snapshots still remain and generate cost
- Unused and outdated snapshots should be **identified and deleted**
- Removing old snapshots helps:
  - Reduce storage cost
  - Keep backup clean and manageable

Example:

- Daily snapshots are taken
- Snapshots older than **30 or 60 days** are not needed
- Delete old snapshots

- Result:
  - Less storage usage
  - **Lower AWS bill**

Best Practices:

- Keep only **latest required snapshots**
- Delete snapshots of:
  - Old projects
  - Terminated instances
- Use **EBS Lifecycle Manager** to:
  - Automatically delete old snapshots
  - Retain only recent backups

—> "Removing old EBS snapshots saves cost because AWS charges for snapshot storage even when the volume or instance is deleted."

c) Choosing the correct EBS volume type helps **reduce storage cost without affecting performance**.

Points:

- `gp2` and `gp3` are **General Purpose SSD** volumes
- `gp3` is **cheaper than gp2**
- `gp3` gives:
  - Better performance control
  - Lower cost per GB
- In `gp2`, performance is linked to volume size
- In `gp3`, performance is **independent of volume size**

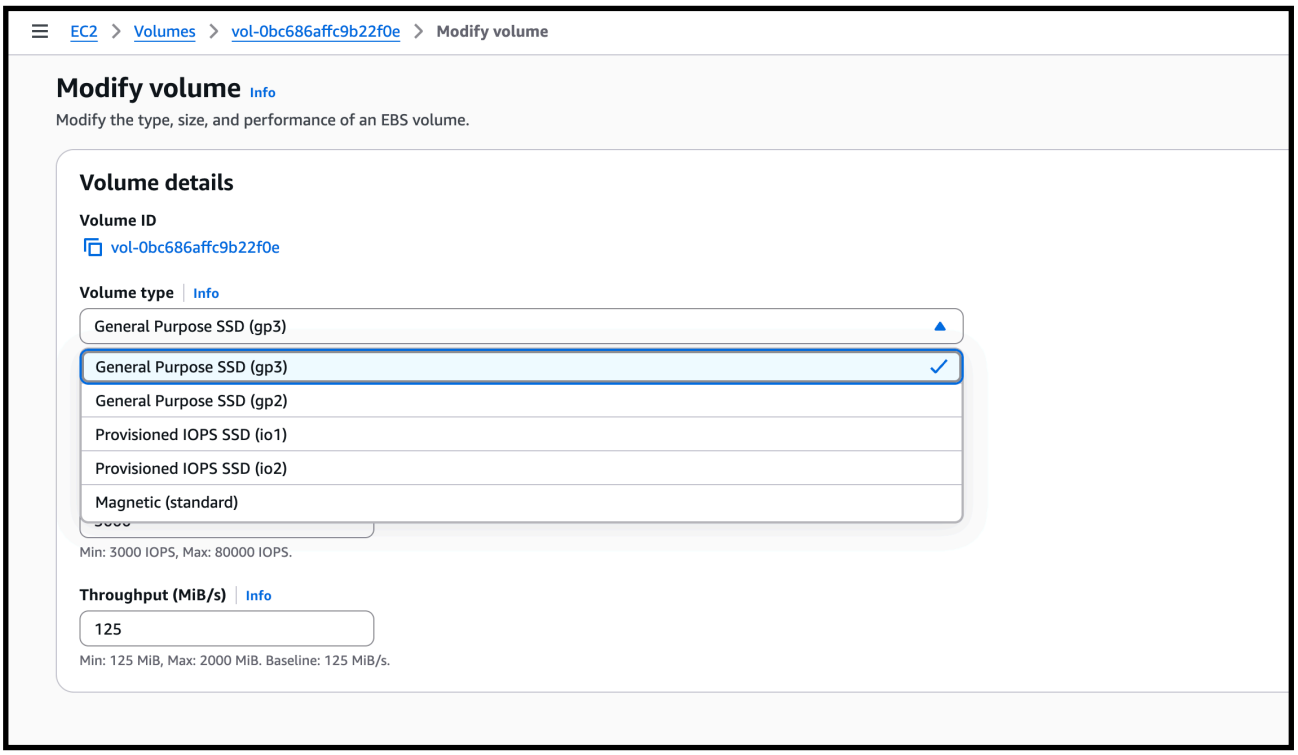
- For most workloads:
  - gp3 provides same or better performance
  - at **lower cost**
- Recommended for:
  - Web servers
  - Application servers
  - Dev and test environm

Example:

- 100 GB volume:
  - gp2 → higher cost
  - gp3 → lower cost with same performance

—-> steps

Go to volume select the volume and—> modify the volume.
—> change the volume type to gp3



Result :

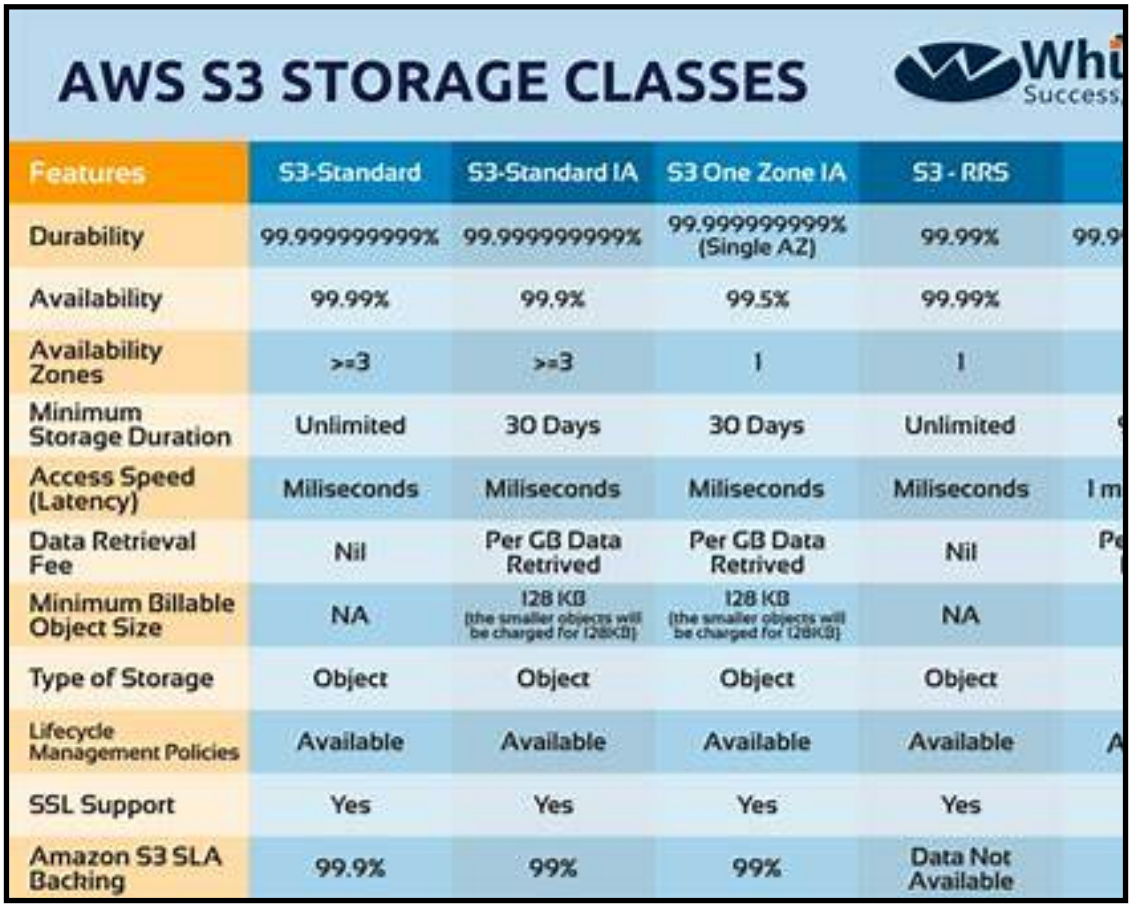
- No impact on application
- **Monthly EBS cost reduced**

Best Practice:

- Use `gp3` as **default volume type**
- Migrate existing `gp2` volumes to `gp3`

—> “Using gp3 instead of gp2 reduces EBS cost because gp3 is cheaper and gives better performance flexibility.”

—> S3 Cost Optimization



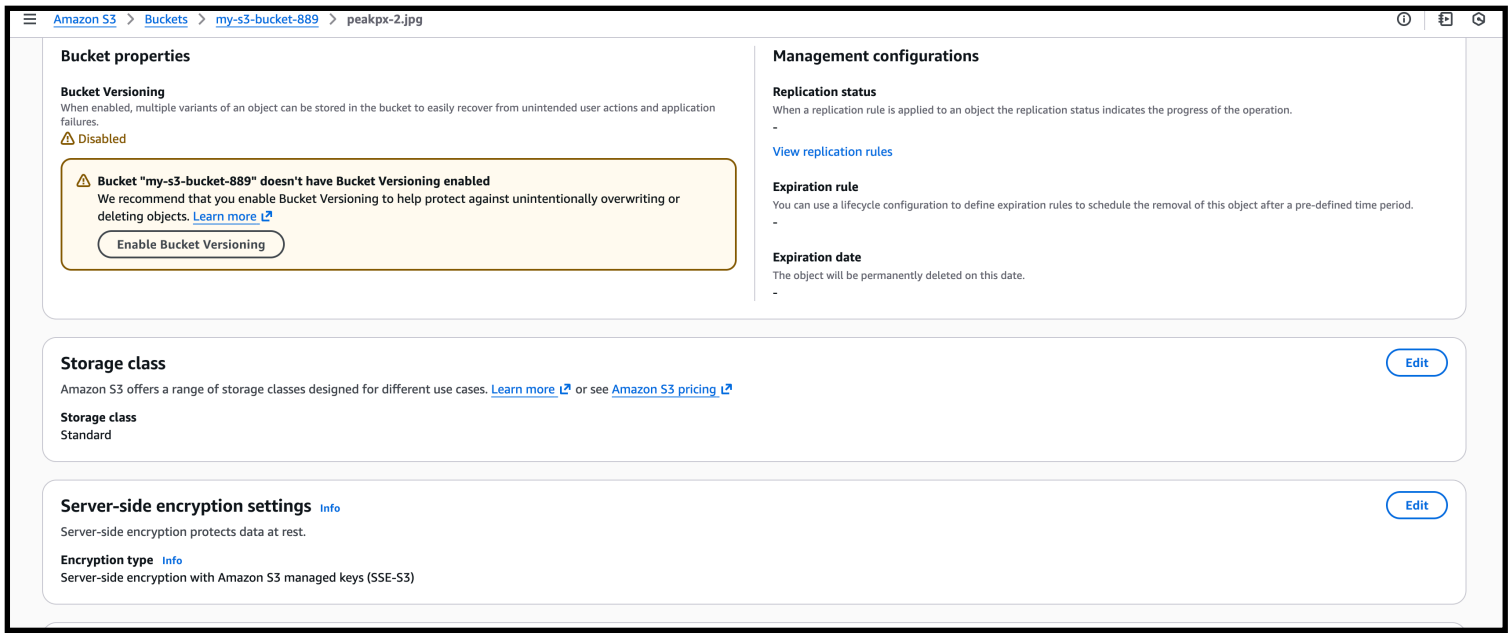
Features	S3-Standard	S3-Standard IA	S3 One Zone IA	S3 - RRS	
Durability	99.999999999%	99.999999999%	99.999999999% (Single AZ)	99.99%	99.9
Availability	99.99%	99.9%	99.5%	99.99%	
Availability Zones	>=3	>=3	1	1	
Minimum Storage Duration	Unlimited	30 Days	30 Days	Unlimited	
Access Speed (Latency)	Miliseconds	Miliseconds	Miliseconds	Miliseconds	I m
Data Retrieval Fee	Nil	Per GB Data Retrived	Per GB Data Retrived	Nil	Pe
Minimum Billable Object Size	NA	128 KB (the smaller objects will be charged for 128KB)	128 KB (the smaller objects will be charged for 128KB)	NA	
Type of Storage	Object	Object	Object	Object	
Lifecycle Management Policies	Available	Available	Available	Available	A
SSL Support	Yes	Yes	Yes	Yes	
Amazon S3 SLA Backing	99.9%	99%	99%	Data Not Available	

—> a) Cost Optimization Action Performed on S3

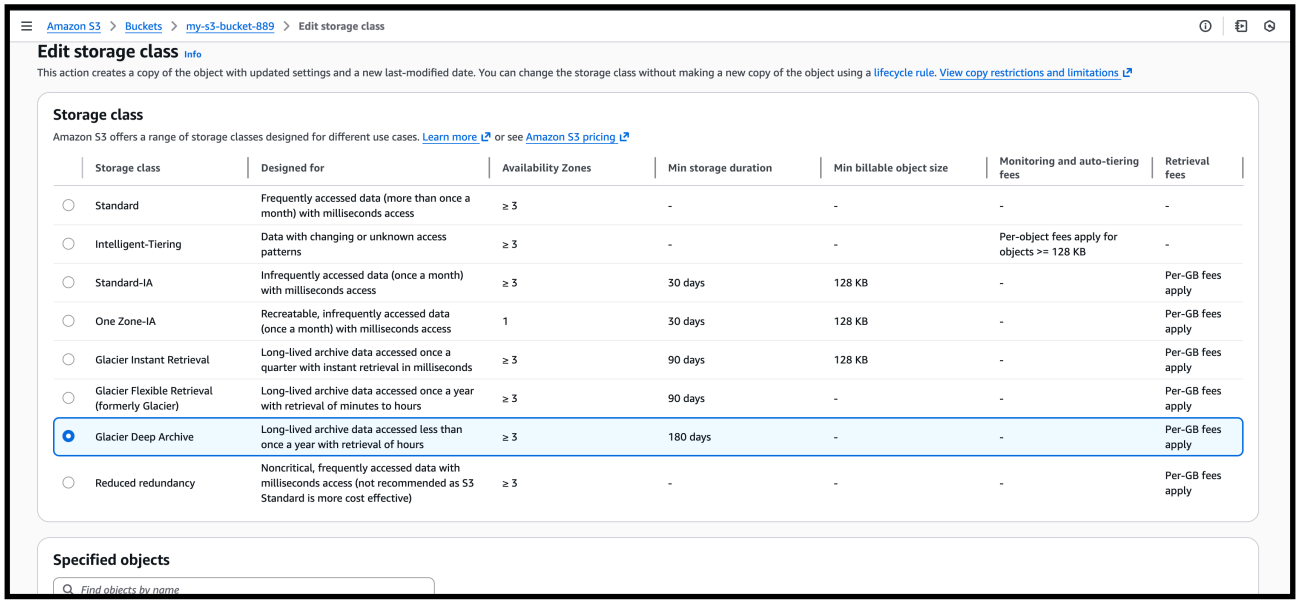
Action: Changed storage class of object to cheaper option

Points:

- Initially, the object was stored in **S3 Standard**
- S3 Standard is used for **frequently accessed data**



- The file was **not accessed regularly**
- Keeping it in S3 Standard was **unnecessarily increasing cost**
- To reduce cost, the storage class was changed to **Glacier Deep Archive**



- Glacier Deep Archive is used for:
  - Long-term storage
  - Rarely accessed data
- After changing the storage class:
  - Storage cost was **significantly reduced**
  - Data is still safely stored
  - Only retrieval time is higher, which is acceptable for archive data

Result:

- No data loss
- Lower S3 monthly bill
- Effective cost optimization achieved

—>“I optimized S3 cost by moving rarely accessed data from S3 Standard to Glacier Deep Archive, which reduced storage cost.”

—> b) Enable Lifecycle Policies (S3 Cost Optimization)

Cost Optimization Action Performed: Enabled Lifecycle Policy

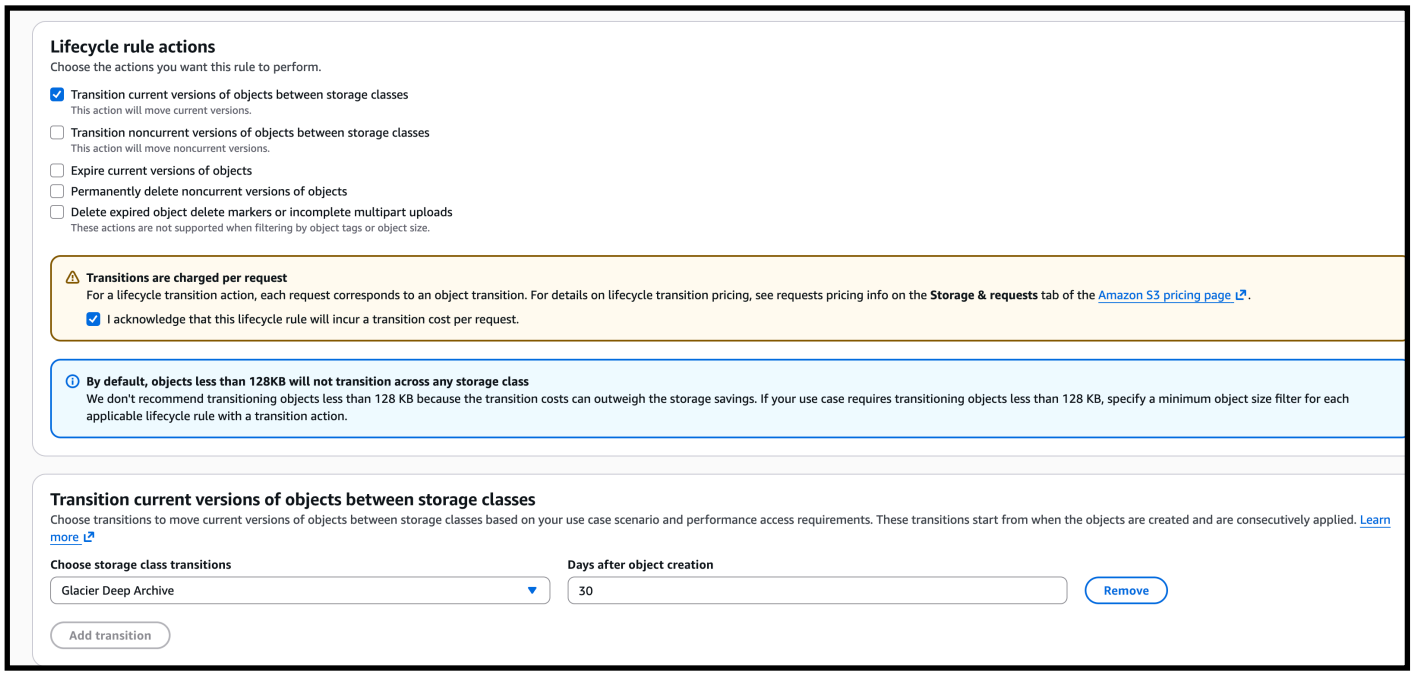
Points:

- Data stored in S3 **keeps increasing over time**
- Old files are **not accessed frequently**
- Keeping old data in **S3 Standard increases cost**
- To optimize cost, an **S3 Lifecycle Policy** was enabled

- Lifecycle policy automatically manages storage class based on age
- No manual work is required after configuration

Example:

- Objects stored in S3 Standard
- Lifecycle rule configured:
  - After **30 days → move data to Glacier**
- Result:
  - Storage automatically moved to cheaper class
  - No data loss
  - Reduced S3 storage cost



S3 Lifecycle Policy – Cost Optimization (Current Versions)

Points:

- Newly uploaded objects are stored in **S3 Standard** for immediate access
- After some time, these objects are **rarely accessed**
- Keeping such data in S3 Standard increases storage cost

- A lifecycle policy was configured to manage this automatically
- The rule transitions **current versions of objects** to **Glacier Deep Archive**
- Transition happens **30 days after object creation**
- Glacier Deep Archive is the **lowest-cost S3 storage class**
- This significantly reduces storage cost for long-term data
- No manual intervention is required after configuration

Result:

- Automatic movement of old data to cheaper storage
- Reduced S3 monthly bill
- Efficient long-term data storage

—> configured an S3 lifecycle rule to move current object versions to Glacier Deep Archive after 30 days to reduce storage cost.

—> AWS KMS Cost Optimization –

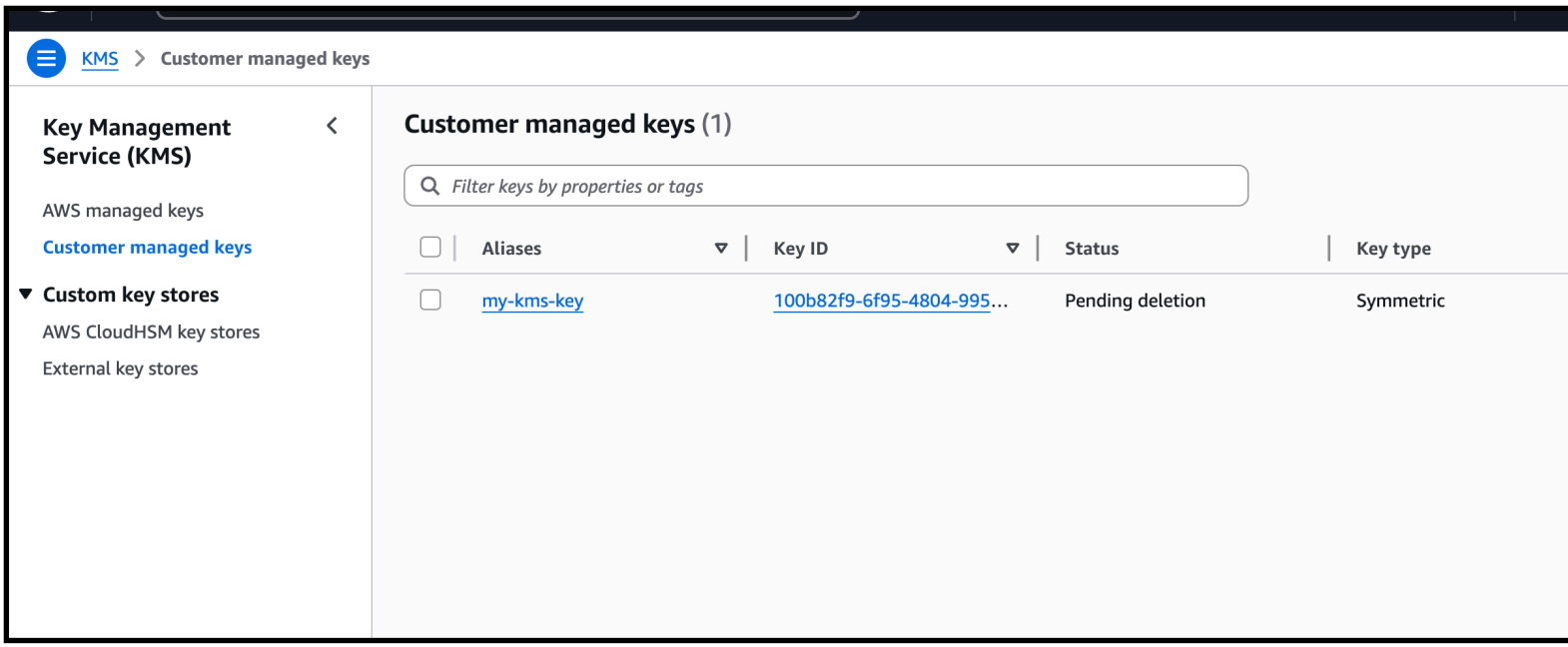
Deleting Unused KMS Keys

Cost Optimization Action: Deleted Unused KMS Key

Points:

- AWS KMS charges **monthly cost for each customer-managed key**
- Some KMS keys are created for:
  - Testing
  - Temporary projects
- After the project is completed, the KMS key is **no longer required**
- Keeping unused KMS keys **continues to generate cost**

- To optimize cost, the **unused KMS key was scheduled for deletion**
- After deletion:
  - AWS **stops charging** for that KMS key
  - Unnecessary security resources are removed



What Happens When a KMS Key Is Deleted

Important Points:

- KMS keys are **not deleted immediately**
- They are **scheduled for deletion** (minimum 7 days, maximum 30 days)
- During the waiting period:
  - Key cannot be used for encryption
  - Key can be restored if needed
- After deletion is completed:
  - Key is **permanently removed**
  - Data encrypted with that key **cannot be decrypted**
- AWS **does not charge** for deleted KMS keys

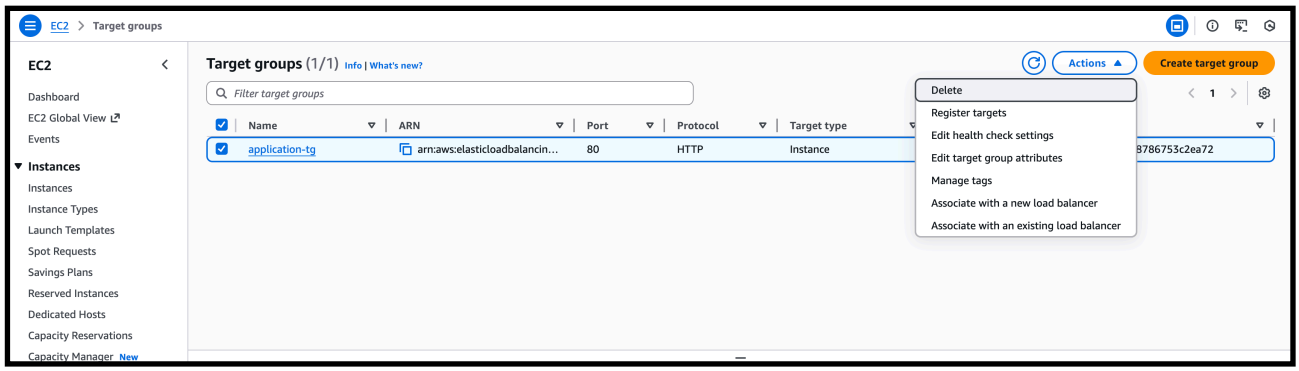
Result (Cost Optimization Outcome)

- Reduced KMS monthly cost
- Removed unused security resources
- Cleaner and cost-efficient AWS environment

—> Cost was optimized by deleting unused KMS keys, which stopped unnecessary monthly KMS charges.

—> Load Balancer Cost Optimization

- Use **Application Load Balancer** only when required
- Delete unused target groups



- Target groups are created for:
  - Load Balancers
  - Testing
  - Old applications
- Over time, some target groups become **unused**
- These target groups:

    - Are **not attached to any Load Balancer**
    - Do not receive traffic
- Keeping unused target groups **creates management overhead**
- To optimize resources, unused target groups were **identified and deleted**
- Deleting unused target groups helps:
    - Clean up AWS environment
    - Avoid confusion during configuration
    - Improve resource management

What Happens After Deleting Target Groups

Points:

- No application impact if target group is **not attached**
- Load Balancer continues to work normally
- Unused backend configurations are removed
- AWS environment becomes **clean and optimized**

—> Unused target groups that were not attached to any load balancer were deleted to clean up resources and improve cost and resource management.

- Remove idle load balancers

—-> Best Practices Summary

—> Delete unused resources
—> Right-size EC2 & RDS
—> Use Auto Scaling
—> Use S3 lifecycle rules
—> Monitor bills daily
—> Set AWS Budgets

Conclusion :

AWS Cost Optimization helps organizations **reduce cloud cost without affecting performance**. By choosing the right service size, removing unused resources, and monitoring usage, we can **save a lot of money in AWS**.